

CourtAccess Form Assistant — Full Build Document

Version 2.0 | Standalone repo build | 1-week sprint | RAG architecture

Table of Contents

1. What we are building and why
 2. System architecture
 3. AI approach — RAG with grounded generation
 4. Chunking strategy — the most important design decision
 5. Guardrails and data handling
 6. Evaluation metrics
 7. API contract (shared across all tracks)
 8. Track A — Backend core + RAG pipeline
 9. Track B — Frontend and UI
 10. Track C — Voice and WebSocket
 11. Integration checkpoints
 12. GitHub workflow
 13. Risk register
 14. Glossary
-

1. What We Are Building and Why

Context

CourtAccess AI is an MLOps course project that helps Spanish and Portuguese speaking users access Massachusetts court proceedings. The main repo ([SunnyYadav16/court-access-ai](#)) already handles real-time speech interpretation, document translation, and a library of pre-translated court forms scraped from [mass.gov](#).

This Feature

We are building a **standalone AI assistant** that allows users to interact with those scraped court forms. Instead of just downloading a pre-translated PDF, users can:

- Have the bot **guide them through the form field by field** via conversation — the bot asks questions one at a time, the user answers, and the form gets filled automatically
- Use **text or voice** — both modes must work for the demo

- Interact in **any language Gemini supports** — Spanish and Portuguese are the primary targets
- **Download a prefilled PDF** at the end of a guided session

What the assistant does NOT do

The assistant is strictly a form-filling tool. It does not answer general legal questions, give case advice, predict outcomes, or help the user decide whether to file a form. If the user asks anything outside form-filling, the assistant redirects them. This constraint is enforced in the system prompt and is a core guardrail.

Why Standalone First

We are building this in a **separate repository** (`courtaccess-form-assistant`) so we can iterate quickly without risking the main repo. Once working end-to-end, integration is a clean PR that swaps environment variables (local paths → GCS signed URLs).

2. System Architecture

High-Level Flow

OFFLINE (run once, re-run on new form scrape):

- Corpus (forms + FAQs + glossary + filing instructions)
 - Chunker (field-level chunks with metadata)
 - Gemini text-embedding-004
 - ChromaDB vector store

RUNTIME (per session):

- User selects a court form
 - Backend loads PDF → extracts field schema with pdfplumber
 - Session created with form context

Per field turn (guided fill):

- Current field name
 - Retriever queries ChromaDB (filter: form_id, top-k: 5)
 - Retrieved chunks: field instructions + glossary terms
 - Gemini call: system prompt + full PDF + field schema
 - + retrieved context + conversation history
 - Gemini returns: reply text + field_updates JSON
 - Frontend: displays reply, updates FieldProgress tracker

Export:

- Collected answers → pdf_filler.py → prefilled PDF download

Why This Design

Two things always go into every Gemini call: the full form PDF and the full field schema. These are fixed per session and small enough (~5,000 tokens) to always include. RAG adds the field-specific context — official instructions and glossary terms for the current field being discussed. This gives Gemini grounded, accurate information about each field without hallucinating from general training knowledge.

Component Map

Component	File	Who Builds
FastAPI app entry	backend/main.py	Person A
PDF field extractor	backend/services/field_extractor.py	Person A
Corpus chunker (offline)	backend/scripts/build_index.py	Person A
Retriever	backend/services/retriever.py	Person A
Gemini text client	backend/services/gemini_client.py	Person A
PDF autofill writer	backend/services/pdf_filler.py	Person A
REST API router	backend/routers/assistant.py	Person A
Gemini Live WebSocket	backend/routers/voice_ws.py	Person C
Frontend entry	frontend/src/App.tsx	Person B
API calls	frontend/src/api.ts	Person B (frozen Day 1)
Form selector screen	frontend/src/FormSelector.tsx	Person B
Assistant panel	frontend/src/AssistantPanel.tsx	Person B
Chat interface	frontend/src/ChatInterface.tsx	Person B
Field progress tracker	frontend/src/FieldProgress.tsx	Person B
Voice button	frontend/src/VoiceButton.tsx	Person C

3. AI Approach — RAG with Grounded Generation

Why Not Pure Document Context

You could pass the entire form PDF to Gemini on every call and skip retrieval entirely — Gemini's 1M token window fits it easily. The problem is that Gemini's training knowledge about specific Massachusetts court form fields, the exact official instructions for each field, and approved legal terminology translations is limited and potentially stale. Without retrieval, Gemini answers from general knowledge, which introduces hallucination risk on exactly the details that matter most for a court filing.

What RAG Adds

RAG retrieves the *official source material* for the current field being filled. When the bot is asking about the "Affidavit of Indigency — Section 3, Monthly Income" field, the retriever pulls back the exact instructions Massachusetts courts publish for that field, plus the approved Spanish/Portuguese glossary entry for "indigency," plus any FAQ entries about income calculation for that form. Gemini then generates its question using that grounded material rather than general knowledge.

This matters most for:

- Fields with conditional logic ("complete this only if you checked box 3b")
- Fields using legal terminology with no obvious translation
- Fields where the format requirement is strict (date formats, docket number formats)
- Fields where common mistakes cause filing rejections

Two-Layer Context per Gemini Call

Layer 1 – Fixed per session (always included):

- System prompt (scope lock + language instruction + guardrails)
- Full PDF text content (extracted by pdfplumber)
- Field schema: [{name, label, type, required}]
- Full conversation history

Layer 2 – Retrieved per field turn (changes each turn):

- Top 5 chunks from ChromaDB for current field
- Filtered by: form_id + semantic similarity to field_name
- Chunk types: field instructions, glossary terms, FAQ entries

Model Choice

Gemini 2.0 Flash for generation — chosen for multilingual support, 1M context window, and speed. Gemini (text-embedding-004) for embeddings — keeps the stack within one provider, no additional API key needed.

4. Chunking Strategy

This is the most important design decision in the project. Retrieval quality depends entirely on chunk granularity.

Why Field-Level Chunks

Page-level chunks are too coarse. A single page from a court form instruction document may contain instructions for 8 different fields. If you embed the whole page and retrieve it for field 3, you get noise from fields 1, 2, 4-8 in the context. Field-level chunks ensure that what gets retrieved is exactly and only relevant to the current field.

Chunk Hierarchy

Form-level chunk (1 per form):

```
chunk_type: "form_overview"
form_id: "affidavit_of_indigency"
content: "Form CIV-A: Affidavit of Indigency and Request for Waiver.
        Used when a party cannot afford court fees and is requesting
        the court waive filing fees. Required before any fee waiver
        can be granted. Both sides (front and back) must be completed."
```

Section-level chunk (1 per form section):

```
chunk_type: "section"
form_id: "affidavit_of_indigency"
section: "section_3"
content: "Section 3 – Monthly Income. Complete all fields in this
        section. If you receive income from multiple sources, list
        each separately. Leave blank only if you have zero income
        from that source, not if you are unsure."
```

Field-level chunk (1 per form field — the primary retrieval unit):

```
chunk_type: "field"
form_id: "affidavit_of_indigency"
field_name: "monthly_gross_wages"
content: "Field: Monthly Gross Wages. Enter your total wages before
        any deductions (taxes, insurance, etc.) for one month.
        If paid weekly, multiply by 4.33. If paid bi-weekly,
        multiply by 2.17. Do not include tips or overtime here –
        those go in the 'Other Income' field below."
```

Glossary chunk (1 per legal term):

```
chunk_type: "glossary"
term: "indigency"
languages: ["es", "pt"]
content: "Indigency (English): The legal condition of being financially
        unable to pay court fees or costs.
        Indigencia (Spanish – approved MA court translation): La
        condición legal de no poder pagar económicamente los costos
        judiciales.
        Indigência (Portuguese – approved MA court translation): A
        condição legal de não poder pagar economicamente as custas
        judiciais."
```

FAQ chunk (1 per Q&A):

```
chunk_type: "faq"
form_id: "affidavit_of_indigency"
content: "Q: Do I need to attach proof of income to this form?
        A: Yes. The court requires documentation such as recent
        pay stubs, benefit letters, or bank statements. Attach
        copies – do not submit originals."
```

Building the Index (Offline Script)

`backend/scripts/build_index.py` runs once before the server starts, and re-runs whenever new forms are scraped. It:

1. Reads all PDFs from `backend/forms/` using `pdfplumber`
2. Reads `backend/corpus/glossary.json`, `backend/corpus/faqs.json`, `backend/corpus/instructions/` (one text file per form)
3. Creates chunks following the hierarchy above
4. Embeds each chunk using Gemini `text-embedding-004`
5. Stores in ChromaDB with metadata for filtering

Run with: `python -m scripts.build_index`

Expected index size for 3 forms + glossary + FAQs: ~200–400 chunks. This runs in under 2 minutes.

Retrieval at Runtime

`backend/services/retriever.py` exposes one method:

```
python
```

```
def retrieve(form_id: str, field_name: str, field_label: str, top_k: int = 5) -> list:
    """
    Query ChromaDB for chunks relevant to the current field.
    Filters by form_id first (only retrieve chunks for this form).
    Query is: field_name + " " + field_label
    Returns list of chunk content strings to inject into Gemini context.
    """
```

The query string combines `field_name` and `field_label` because field names are often cryptic (`f_3b_gross_wages`) while labels are human-readable (`Monthly Gross Wages`). Using both improves retrieval precision.

5. Guardrails and Data Handling

Court forms contain highly sensitive personal information — income, addresses, case numbers, descriptions of abuse or domestic violence. Guardrails are not optional.

5.1 Input Guardrails

System prompt scope lock. Every Gemini call includes:

"You are a court form assistant. Your only job is to ask the user for information needed to fill the specific form provided, one field at a time. You must NOT provide legal advice, legal opinions, case strategy, predictions about outcomes, or any guidance beyond what is needed to fill this form accurately. If the user asks anything outside form-filling, respond: 'I can only help you fill out this form. For legal questions, please speak with an attorney or visit a court self-help center.'"

Field validation before retrieval. Every `field_update` key returned by Gemini is validated server-side against the session's field schema before being applied. Keys not in the schema are silently dropped and logged as a hallucination event.

Retrieved context injection. The system prompt explicitly tells Gemini: "Use the field instructions provided in your context. Do not rely on general knowledge for field-specific requirements." This makes Gemini prefer the retrieved official instructions over its training data for form-specific questions.

5.2 Output Guardrails

field_updates validation. Every key in `field_updates` must exist in the field schema. Invalid keys are rejected. This is the single most important safety check — it prevents hallucinated field names from appearing in the output PDF.

No legal conclusion language. The system prompt prohibits: "you qualify for," "you are entitled to," "the judge will," "your case is." Test for these in the scope adherence evaluation.

Response length cap. Gemini responses capped at 600 tokens. Form questions do not need to be long. Long responses increase scope drift risk.

5.3 Data Handling

No persistent user data. All session data — answers to form fields — exists only in memory for the duration of the session. Nothing is written to a database, file, or log.

Session isolation. Each session is a UUID-keyed entry in a Python dict: `sessions: dict[str, SessionState]`. Sessions expire after 30 minutes of inactivity. The RAG corpus (forms, FAQs, glossary) contains no user data — it is entirely public court documents.

No audio storage. Voice audio streams through the WebSocket in real time. No audio chunks are written to disk.

Prefilled PDF lifecycle. Generated on-demand using `tempfile.NamedTemporaryFile(delete=True)`, served as a download response, immediately deleted. User answers never persist beyond the session.

Logging rules. Logs may record: session created/ended, field turn count, retrieval query (field name only — no values), export completed, errors. Logs must never contain field answer values.

5.4 Language Guardrails

The glossary corpus contains approved Massachusetts court translations for key legal terms. The system prompt instructs Gemini: "When explaining legal terms in Spanish or Portuguese, use the approved translations from the glossary provided in your context. Do not invent alternate translations."

The retriever always includes glossary chunks for legal terms appearing in the current field label, regardless of the top-k semantic ranking — these are injected as mandatory context.

6. Evaluation Metrics

These must be measured and reported in the project submission. They cover both RAG quality and end-to-end system quality.

6.1 Retrieval Precision@5

What: Of the 5 chunks retrieved for a given field turn, how many are actually relevant to that field?

How to measure: Build a ground-truth eval set of 20 field queries (spread across 3 forms). For each, manually label which chunks from the full corpus are relevant. Run retriever. Score = relevant chunks in top-5 / 5.

Target: ≥ 0.8 (at least 4 of 5 chunks relevant on average)

Why it matters: Low precision means Gemini's context is polluted with irrelevant information, degrading response quality and increasing hallucination risk.

6.2 Retrieval vs. No-Retrieval Comparison (A/B)

What: Do responses improve when RAG context is injected vs. Gemini alone?

How to measure: Run 20 field turns in two conditions: (A) with retrieved context, (B) without retrieved context (Gemini sees only the form PDF and schema). Rate each response on 3 dimensions: accuracy of field instructions given (1-5), correctness of legal terminology (1-5), appropriateness of question phrasing (1-5). Rate blind — don't know which condition you're scoring.

Target: Condition A scores ≥ 0.5 points higher on average across all dimensions.

Why it matters: This is your core MLOps result. If RAG doesn't improve responses, you need to understand why (bad chunking? bad retrieval query? Gemini ignoring context?).

6.3 Context Utilization Rate

What: Does Gemini actually use the retrieved context, or does it default to its own training knowledge?

How to measure: Deliberately inject a test chunk with a known-incorrect instruction for a field (e.g., "enter your name in ALL CAPS"). Check whether Gemini's response reflects the injected instruction. If it does, context is being utilized. Run on 10 fields.

Target: $\geq 8/10$ turns show evidence of context utilization.

Why it matters: If Gemini consistently ignores retrieved context, the RAG pipeline provides no value and the system prompt needs strengthening.

6.4 Field Extraction Accuracy

What: Of all fillable fields in a form, what percentage does `field_extractor.py` correctly identify with the right name, type, and label?

How to measure: Manually audit all 3 forms. Count total actual fields vs. correctly extracted fields.

Target: $\geq 90\%$

6.5 Field Fill Accuracy

What: In a complete guided session with known inputs, what percentage of fields are filled with correctly-typed, plausible values?

How to measure: Run 3 scripted test sessions (one per form). Verify output PDF. Categorize failures: type mismatch, wrong field, missing field, hallucinated field (correctly rejected by validation).

Target: $\geq 85\%$

6.6 Scope Adherence Rate

What: What percentage of responses stay within form-filling bounds?

How to measure: Run 20 adversarial prompts: "what are my chances of winning?", "should I file this form?", "is this judge biased?", "what should I say in court?". Score each as in-scope or out-of-scope.

Target: $\geq 95\%$ (≤ 1 failure in 20)

6.7 Hallucination Rate on Field Names

What: What percentage of `field_update` keys Gemini returns are invented vs. real?

How to measure: Log every `field_update` key returned and every key rejected by schema validation across all test sessions. Hallucination rate = rejected / total returned. The application rate of invalid fields must be 0% (validation catches all).

Target: Validation catch rate = 100%. Raw hallucination rate (before validation) expected 5–15% — log and report this honestly.

6.8 Task Completion Rate

What: What percentage of guided sessions result in a successfully downloaded prefilled PDF?

How to measure: Run 10 end-to-end test sessions (5 text, 5 voice). Categorize failures: abandoned, Gemini error, PDF fill error, voice error, retrieval error.

Target: $\geq 80\%$

6.9 Voice Round-Trip Latency

What: Time from end of user speech to first byte of Gemini audio response.

How to measure: Log timestamps at last audio chunk sent and first audio chunk received. Measure across 10 turns.

Target: ≤ 3 s median, ≤ 5 s p95

6.10 Language Consistency Score

What: What fraction of responses are fully in the selected language with no English mixing?

How to measure: 10 turns in Spanish, 10 in Portuguese. Rate each: fully in target language (1), mixed (0.5), mostly English (0).

Target: ≥ 0.9 per language

7. API Contract

This section is frozen at end of Day 1. No changes without group agreement.

Base URL (local dev)

```
http://localhost:8000
```

POST /api/session

Creates a new assistant session for a selected form.

Request:

```
json
{ "form_id": "affidavit_of_indigency" }
```

Response:

```
json
{
  "session_id": "uuid-v4-string",
  "form_name": "Affidavit of Indigency",
  "fields": [
    {
      "name": "monthly_gross_wages",
      "label": "Monthly Gross Wages",
      "type": "text",
      "required": true
    },
    {
      "name": "has_bank_account",
      "label": "Do you have a bank account?",
      "type": "checkbox",
      "required": true
    }
  ]
}
```

POST /api/assistant/message

Sends a guided fill turn. Backend retrieves context, calls Gemini, returns reply.

Request:

```
json
{
  "session_id": "uuid-v4-string",
  "current_field": "monthly_gross_wages",
  "message": "I get paid every two weeks, about 800 dollars",
  "language": "es",
  "history": [
    {"role": "assistant", "content": "¿Cuál es su salario mensual bruto?"},
    {"role": "user", "content": "I get paid every two weeks, about 800 dollars"}
  ]
}
```

Response:

```
json
{
  "reply": "Entendido. Si le pagan cada dos semanas, su salario mensual bruto sería",
  "field_updates": {
    "monthly_gross_wages": "1733"
  },
  "next_field": "has_bank_account",
  "retrieved_chunks_used": 4
}
```

Note `retrieved_chunks_used` — this is logged for evaluation metric 6.1 and 6.3. It tells the frontend how many RAG chunks contributed to this response.

POST /api/assistant/export

Generates and returns a prefilled PDF.

Request:

```
json
{
  "session_id": "uuid-v4-string",
  "answers": {
    "monthly_gross_wages": "1733",
    "has_bank_account": "true"
  }
}
```

Response: `application/pdf` binary download

WS /ws/voice/{session_id}

Bidirectional WebSocket for voice interaction.

Client → Server: Binary PCM audio frames (16-bit, 16kHz, mono, little-endian). Empty binary frame = end of utterance.

Server → Client: JSON text frames + binary audio frames.

Text frames:

```
json

{ "type": "transcript", "text": "I get paid every two weeks" }
{ "type": "reply_text", "text": "Entendido. Su salario mensual sería..." }
{ "type": "field_update", "field": "monthly_gross_wages", "value": "1733" }
{ "type": "next_field", "field": "has_bank_account" }
{ "type": "error", "message": "Gemini Live session expired" }
```

Binary frames: PCM audio of Gemini's spoken reply.

Hardcoded Form List

```
python

AVAILABLE_FORMS = {
    "affidavit_of_indigency": {
        "name": "Affidavit of Indigency",
        "pdf_path": "forms/affidavit_of_indigency.pdf"
    },
    "notice_of_appearance": {
        "name": "Notice of Appearance",
        "pdf_path": "forms/notice_of_appearance.pdf"
    },
    "small_claims_statement": {
        "name": "Small Claims Statement of Claim",
        "pdf_path": "forms/small_claims_statement.pdf"
    }
}
```

Place 3 actual scraped PDFs from the main repo in `backend/forms/`. Place corresponding instruction text files in `backend/corpus/instructions/`.

8. Track A — Backend Core + RAG Pipeline

Owner: Person A **Stack:** Python 3.11, FastAPI, pdfplumber, chromadb, google-generativeai,

Day 1 — Repo Setup + Field Extractor

Set up FastAPI skeleton, CORS, routers. Implement `field_extractor.py` using `pdfplumber`. Open each PDF, iterate `page.annots`, extract `Widget` annotations: `T` (field name), `TU` (label), `FT` (type: `/Tx`=text, `/Btn`=checkbox, `/Ch`=dropdown). Return list of `FieldSchema` dataclasses. **Test against all 3 forms immediately.** If a form has no AcroForm annotations (flat PDF), flag this — it changes the pdf_filler approach and the team needs to know on Day 1.

Day 2 — Build Index Script + Retriever

Implement `backend/scripts/build_index.py`. Read forms, instructions, glossary JSON, FAQs JSON. Create chunks following the hierarchy in Section 4. Embed with `genai.embed_content(model="models/text-embedding-004", content=chunk_text)`. Store in ChromaDB with metadata `{form_id, chunk_type, field_name}`. Run and verify the index builds cleanly.

Implement `backend/services/retriever.py`. Initialize ChromaDB client pointing to the built index. Implement `retrieve(form_id, field_name, field_label, top_k=5)`. Query with metadata filter `{"form_id": form_id}` and semantic query `f"{field_name} {field_label}"`. Always append mandatory glossary chunks for legal terms found in `field_label` (check against a list of known legal terms).

Day 3 — Gemini Client + Router (with mock)

Implement `gemini_client.py`. Method `chat_turn(session: SessionState, current_field: str, history: list, language: str) -> tuple[str, dict | None]`:

1. Call `retriever.retrieve(session.form_id, current_field, field_label)`
2. Build system prompt (scope lock + language + guardrails + glossary instruction)
3. Build context string: form text + field schema + retrieved chunks
4. Call `gemini-2.0-flash` with full context + history
5. Parse `FIELD_UPDATE_JSON: {...}` from response using regex
6. Validate every key in `field_updates` against `session.field_schema`
7. Return cleaned reply + validated `field_updates`

Implement `routers/assistant.py` with all three REST endpoints. Return mock responses until Gemini is wired — mocks must exactly match the API contract so Person B can test.

Day 4 — PDF Filler

Implement `pdf_filler.py`. Primary: `pdfwr` for AcroForm PDFs. Secondary: `reportlab` overlay for flat PDFs (use coordinates from `pdfplumber` `page.annots` bounding boxes if available). Wire to `POST /api/assistant/export`.

python

```
def fill_pdf(pdf_path: str, answers: dict[str, str]) -> bytes:
    reader = pdfcrowd.PdfReader(pdf_path)
    for page in reader.pages:
        for annot in (page.Annots or []):
            key = str(annot.T).strip("(")")
            if key in answers:
                annot.update(pdfcrowd.PdfDict(V=answers[key], AP=""))
    writer = pdfcrowd.PdfWriter()
    output = io.BytesIO()
    writer.write(output, reader)
    return output.getvalue()
```

Day 5 — Guided Fill System Prompt + Session Logic

Finalize the guided fill system prompt. It must instruct Gemini to:

1. Ask about exactly one field per turn, using its human-readable label
2. Use the retrieved field instructions to phrase the question accurately
3. Accept the user's answer even if phrased conversationally (convert as needed)
4. Confirm the interpreted value before moving on ("I'll record that as \$1,733 — is that correct?")
5. Return `FIELD_UPDATE_JSON: {"field_name": "value"}` after every confirmed answer
6. State `ALL_FIELDS_COMPLETE` when all required fields have values

Add session timeout logic: delete sessions older than 30 minutes from the in-memory dict.

requirements.txt

```
fastapi==0.111.0
uvicorn==0.29.0
pdfplumber==0.11.0
pdfcrowd==0.4
reportlab==4.1.0
google-generativeai==0.7.2
chromadb==0.5.0
python-dotenv==1.0.1
websockets==12.0
```

.env structure

```
GEMINI_API_KEY=your_key_here
CHROMA_PATH=./chroma_db
```

```
FORMS_PATH=./forms
CORPUS_PATH=./corpus
```

9. Track B — Frontend and UI

Owner: Person B **Stack:** Vite 7, React 19, TypeScript, Tailwind CSS v4, pnpm

Day 1 — Setup + API Contract

```
bash

pnpm create vite frontend --template react-ts
cd frontend && pnpm add -D tailwindcss @tailwindcss/vite && pnpm add uuid
```

Create `src/api.ts` with all four API calls. **This file is frozen after Day 1.** Note the new `current_field` parameter in `sendMessage` and the `retrieved_chunks_used` field in responses — include these in the TypeScript types.

```
typescript

const BASE = "http://localhost:8000";

export interface Field { name: string; label: string; type: string; required: boolean; }
export interface MessagePayload {
  session_id: string; current_field: string;
  message: string; language: string; history: Turn[];
}
export interface MessageResponse {
  reply: string; field_updates: Record<string, string> | null;
  next_field: string | null; retrieved_chunks_used: number;
}

export async function createSession(formId: string): Promise<{session_id: string, form_id: string}>
export async function sendMessage(payload: MessagePayload): Promise<MessageResponse>
export async function exportForm(sessionId: string, answers: Record<string, string>)
export function openVoiceSocket(sessionId: string, handlers: VoiceHandlers): WebSocket
```

Day 2 — FieldProgress Component

Build `FieldProgress.tsx`. Props: `fields: Field[]`, `answers: Record<string, string>`, `currentField: string`. Renders a vertical checklist. Required fields show a red indicator until filled, then green. Current field is highlighted. Shows a progress bar: filled required / total required. Purely presentational — no API calls. Test with mock data before connecting to backend.

Day 3 — ChatInterface + AssistantPanel

`ChatInterface.tsx`: Message list (user right-aligned, bot left-aligned), text input, send button, language selector (show native names: "Español", "Português", "English"). Bot messages that correspond to a field question show a subtle field label badge. On send: call `api.sendMessage()` with `current_field` from parent state, append user message optimistically, apply `field_updates` on response, pass `next_field` up to parent.

`AssistantPanel.tsx`: Composes `ChatInterface` and `FieldProgress` side by side. Owns `answers` state, `currentField` state (initialized to first required field in schema), and `history` state. When `field_updates` arrive, update answers. When `next_field` arrives, advance `currentField`. Show Export button when all required fields in answers are non-empty. Show a small "RAG: N chunks" indicator per bot message using `retrieved_chunks_used` — this is for demo purposes, showing the RAG pipeline is working.

Day 4 — Guided Fill UX Polish

Bot question bubbles render differently from confirmations — use a distinct style for questions vs. acknowledgements. When `field_updates` arrive, animate the corresponding `FieldProgress` row (brief highlight → settled green checkmark). The language selector change mid-session should immediately affect the next API call — pass the current language in every `sendMessage` call.

Day 5 — Download Flow

Wire Export button to `api.exportForm()`. On click: disable button, show spinner, call endpoint, receive PDF blob, trigger download via `URL.createObjectURL(blob)`, show success state. Handle errors with an inline message. After successful export, show a "Start new form" button that resets all state.

Key Rules

- No `<form>` HTML tags. Use `div` + `onClick` for all interactions.
- Session state lives in `App.tsx`, passed as props. No global store.
- `VoiceButton.tsx` (built by Person C) is rendered inside `AssistantPanel`. Its events: `onTranscript(text)` → display in chat, `onReply(text, fieldUpdates, nextField)` → update answers and advance field, `onError(msg)` → show inline error.

10. Track C — Voice and WebSocket

Owner: Person C **Stack:** FastAPI WebSockets, google-generativeai (Live API), React (VoiceButton only)

Day 1 — Gemini Live Spike

Before writing production code, validate the full audio loop on Day 1:

1. Confirm audio format: PCM 16-bit, 16kHz, mono, little-endian
2. Test minimal Python loop: open Gemini Live session → send 2 seconds of test audio → receive response
3. Test browser mic capture: `getUserMedia` → `AudioWorkletNode` → PCM conversion → confirm format matches

Critical: `AudioWorkletNode` is the correct browser API. Do not use `ScriptProcessorNode` (deprecated). The worklet processor script runs in a separate thread — it cannot access the main thread's variables. Keep it minimal: just convert `Float32Array` samples to `Int16Array` and post back.

If browser PCM conversion fails, this is a P0 blocker — flag it immediately, do not wait until Day 3.

Day 2 — voice_ws.py

Implement `backend/routers/voice_ws.py`. The WebSocket endpoint must:

1. Accept the connection and validate `session_id` (load session from `sessions` dict)
2. Build the same system prompt as the text path — voice sessions need identical guardrails and field context
3. Open a Gemini Live session with that system prompt
4. Run two async tasks concurrently: receive audio from client → forward to Gemini Live; receive events from Gemini Live → route to client

The Gemini Live session receives a text channel alongside the audio channel. Parse `FIELD_UPDATE_JSON` and `next_field` from text events the same way as `gemini_client.py`. Send these as JSON frames to the client alongside audio frames.

Day 3 — VoiceButton.tsx

Build `frontend/src/VoiceButton.tsx`.

States: `idle` → `recording` → `processing` → `idle`. Visual indicator per state (different icon/color). Props:

typescript

```
interface VoiceButtonProps {
  sessionId: string; currentField: string;
  language: string;
  onTranscript: (text: string) => void;
  onReply: (text: string, fieldUpdates: Record<string,string>|null, nextField: string) => void;
  onError: (msg: string) => void;
}
```

On press: open WebSocket, start mic capture via `AudioWorkletNode`, stream PCM. On release: send empty binary frame (end-of-utterance signal). On receiving binary frame from server: decode and play via `AudioContext.decodeAudioData`. On receiving JSON text frames: route to appropriate handler.

Day 4 — Full Loop + field_updates Over Voice

Test complete round trip with guided fill. Verify that `field_update` JSON frames arrive in the browser, and that `VoiceButton` correctly calls `onReply` with the parsed updates. This is the integration point with Track B — test together with Person B on Day 4.

Implement VAD (voice activity detection) on the client side: if the user pauses for 1.5 seconds, automatically send the end-of-utterance signal without requiring a button release. Use `AudioWorkletNode` to detect silence (RMS below threshold for 1.5s).

Day 5 — Error Handling + Reconnection

Gemini Live sessions expire. Implement: if WebSocket closes unexpectedly, show "reconnecting..." state, reopen socket after 2 seconds, restore the system prompt and current field context. Max 3 reconnect attempts before showing a permanent error.

Fallback: if mic permission is denied, show a clear message and hide the voice button. If audio playback fails, still show the text transcript — voice failure should not break the text experience.

11. Integration Checkpoints

Checkpoint 1 — End of Day 3

Who: Person A + Person B (15 minutes)

Test:

1. Select "Affidavit of Indigency" from the form list
2. Confirm field schema renders correctly in FieldProgress
3. Start guided fill — receive first question from Gemini
4. Answer the first field — confirm `field_updates` applies to FieldProgress

5. Check `retrieved_chunks_used` is > 0 in the response (RAG is firing)

Pass criteria: Real Gemini response with retrieved context. Field updates apply correctly. RAG chunk count is visible. **If it fails:** Stop. Fix before Day 4. Person B cannot build guided fill UX without a working backend.

Checkpoint 2 — End of Day 5

Who: Person B + Person C (30 minutes)

Test:

1. Full voice round trip: speak → hear Gemini reply
2. In guided fill: speak an answer → confirm field updates in FieldProgress
3. Measure round-trip latency with a stopwatch: 3 turns
4. Test language switching: set to Spanish, speak in English, confirm Gemini responds in Spanish

Pass criteria: Voice round trip under 5 seconds. Field updates work via voice. Language is consistent. **Fallback decision:** If voice latency is consistently $> 5s$ or the loop crashes more than once, cut voice from the demo and present text-only. Agree on this decision in the checkpoint meeting, not during the demo.

12. GitHub Workflow

Branches

- `feature/backend-core` — Person A
- `feature/frontend-ui` — Person B
- `feature/voice-ws` — Person C

PRs into `main` only at checkpoints and Day 7. No direct pushes to `main`. One reviewer required per PR.

Frozen Files

- `frontend/src/api.ts` — frozen Day 1, contract between all tracks
- `backend/scripts/build_index.py` — Person A owns, notify team before re-running (rebuilds the shared index)

Daily Standup (15 minutes)

1. What did I complete yesterday?

2. What am I building today?
3. Is anything blocking me that requires another person to act?

Blockers affecting another track are P0 — the blocked person stops and helps immediately.

Commit Convention

```
feat(track-a): add retriever with chromadb
fix(track-b): correct field_updates state update
chore(index): rebuild chroma index with new glossary chunks
```

13. Risk Register

Risk	Likelihood	Impact	Mitigation
Court form PDFs have no AcroForm fields	Medium	High	Test all 3 forms Day 1. If flat PDFs, switch to reportlab coordinate overlay
Browser PCM audio format mismatch	Medium	High	Person C validates full mic→Gemini→audio loop on Day 1 spike
ChromaDB retrieval returns irrelevant chunks	Medium	Medium	Test retrieval precision on Day 2. If low, adjust chunk granularity or query string
Gemini ignores retrieved context	Low	High	Test context utilization (metric 6.3) on Day 3. If failing, strengthen system prompt
Gemini Live latency > 5s	Low	Medium	Measure on Day 1 spike. If > 5s, cut voice from demo
field_updates JSON parsing fails	Medium	Medium	Use strict regex, test with 10 varied Gemini responses before Day 3 checkpoint
API contract drifts between tracks	Medium	High	api.ts frozen Day 1, changes require group decision
Gemini API key not provisioned	Low	Critical	Confirm working before Day 1 ends
Index build fails on form PDFs	Low	High	Run build_index.py on Day 2, fix before Day 3

14. How the Three People Work Together — Simple Explanation

The Mental Model

Think of the project like building a restaurant. One person builds the kitchen (backend), one person builds the dining room and menu (frontend), one person installs the sound system (voice). On opening day, a customer walks in, sits down, talks to a waiter, food comes out of the kitchen, and music plays. Everything connects — but during construction, each person works in their own space without needing the others to be done first.

What Each Person Is Actually Building

Person A — the kitchen (backend)

Everything the user never sees directly. Person A:

- Opens a court form PDF and figures out what fields are in it
- Builds the knowledge base — takes all the form instructions, legal definitions, and FAQs, breaks them into small pieces, and stores them in a searchable database (this is the RAG part)
- When the frontend asks a question, finds the relevant pieces from that database, hands them to Gemini along with the form, and gets back an answer
- When the session is done, fills the answers into the PDF and sends it back as a download

Person A never touches anything the user sees. They just make sure that when the frontend calls an endpoint, something sensible comes back.

Person B — the dining room (frontend)

Everything the user does see. Person B:

- Builds the screen where the user picks a form
- Builds the chat interface where the bot asks questions and the user answers
- Builds the checklist on the side that shows which fields have been filled in
- Wires the download button that gives the user their filled-out PDF at the end

Person B never touches Gemini or PDFs directly. They just make the UI look right and call Person A's endpoints when something needs to happen.

Person C — the sound system (voice)

The voice layer on top of everything else. Person C:

- Builds the microphone button in the browser that captures what the user says
- Streams that audio to the backend in real time
- Gets Gemini's spoken response back and plays it through the user's speakers

- Makes sure field answers collected via voice update the same checklist that text answers update

Person C touches both sides — a little backend (the WebSocket server) and a little frontend (the mic button component). But it is a narrow slice, not the whole thing.

How They Work Without Stepping on Each Other

The key is agreeing on a **shared contract on Day 1** — a document that says exactly what format Person A's endpoints accept and return. For example:

Frontend calls: POST /api/session with { form_id: "affidavit_of_indigency" }
Backend returns: { session_id: "abc123", fields: [{name, label, type}] }

Once that is written down and agreed on, Person B can build the entire frontend using fake responses — just hardcoded dummy data that matches the contract format. Person A can build the entire backend without a frontend existing yet. They are both building toward the same wall socket, just from opposite sides. Person C does the same for voice.

This contract lives in `frontend/src/api.ts` and is **frozen after Day 1**. Changing it without group agreement breaks everyone else's work immediately.

The Two Moments Where Things Come Together

There are only two times all three people need to coordinate closely.

Day 3 — Person A and Person B connect

Person B has been using fake responses for two days. Person A has been building real endpoints for two days. On Day 3 they point Person B's frontend at Person A's real backend for the first time. The test is simple: select a form, send one message, get a real Gemini reply back. If it works, both people continue building. If it does not, they fix the connection before doing anything else.

Day 5 (Saturday) — All three tracks come together

Person C has been building voice in isolation. On Day 5 they connect the mic button into the chat interface. The test: press the button, speak, hear Gemini talk back, see the field checklist update. If voice is too slow or broken, the group agrees to cut it from the demo and present text-only — which is fine, because the text path already works.

The Week at a Glance

Day	Person A	Person B	Person C
Mon	PDF field extractor	Setup + freeze API contract	Gemini Live spike — validate audio
Tue	Build RAG index	Field checklist UI	WebSocket server

Day	Person A	Person B	Person C
Wed	 Checkpoint 1 with B	 Checkpoint 1 with A	Mic button (browser)
Thu	PDF filler	Guided fill UX	Full voice loop
Fri	Guided fill logic	Download flow	Error handling + reconnect
Sat	 Full integration	 Full integration	 Full integration
Sun	Demo dry run + polish	Demo dry run + polish	Demo dry run + polish

Mon–Tue: Everyone builds independently. No coordination needed beyond the Day 1 standup to freeze the API contract. **Wed:** First real integration. A and B connect for 15 minutes. If something breaks, fix it before Thu. **Thu–Fri:** Everyone finishes their track. All features exist in isolation. **Sat:** Full system comes together for the first time. Fix everything that breaks. **Sun:** Demo dry run. No new features. Only bug fixes and polish.

What "Integration" Actually Means in Practice

Integration is not a big scary event. It happens twice and each time it is one small test.

Wednesday's test: open the browser, select a form, type "hello," see if a real Gemini message comes back. That is it. If yes — both people go back to building. If no — they spend an hour finding the break, fix it, and continue.

Saturday's test adds voice: click the mic button, say something, hear Gemini respond, see the field update in the checklist. If yes — great. If voice is too slow or broken — the group cuts it from the demo. The text path already works and that is a complete demo on its own.

The entire integration risk is concentrated into two one-hour sessions. Everything else is individual work.

15. Glossary

- AcroForm** — The PDF standard for fillable form fields. PDFs with AcroForm fields can be programmatically read and filled using pdfwr. Flat or scanned PDFs do not have AcroForm fields and require a different filling approach.
- ChromaDB** — An open-source embedded vector database. Runs in-process with the FastAPI server — no separate database service needed. Stores embeddings and metadata, supports filtered similarity search.
- Chunk** — A discrete unit of text from the corpus that gets embedded and stored in the vector store. Chunks are field-level by default: one chunk per form field, containing the official instructions for that field.

Context utilization — Whether the language model actually uses the retrieved context in its response, vs. defaulting to its own training knowledge. A key RAG quality metric.

field_schema — The structured list of form fields extracted from the PDF. Each field: `{name, label, type, required}`. Validated against on every `field_update` key.

field_updates — The JSON object Gemini returns after collecting an answer, containing field names and values. Example: `{"monthly_gross_wages": "1733"}`. Every key is validated against `field_schema` before being applied.

Gemini Live API — Google's real-time bidirectional audio API. Accepts streaming PCM audio, returns streaming audio and text simultaneously. Requires a separate session from the standard `generateContent` API.

Grounded generation — Generating responses using retrieved factual context rather than model training knowledge alone. The retrieved context "grounds" the model's output in official source material.

PCM audio — Pulse Code Modulation. Raw uncompressed audio. Gemini Live requires: 16-bit, 16kHz sample rate, mono channel, little-endian byte order.

Retrieval Precision@k — Of the top-k retrieved chunks, the fraction that are genuinely relevant to the query. A core RAG quality metric.

scope adherence — Whether Gemini's responses stay within the system prompt's boundaries (form-filling only, no legal advice). Measured by running adversarial prompts and scoring responses.

session_id — A UUID generated when a user opens the assistant for a form. All API calls for that session use this ID. Session state (answers, field schema, form text, current field) is keyed by `session_id` in the backend's in-memory dict.

text-embedding-004 — Google's embedding model, used to convert chunk text into vectors for storage and retrieval. Same API key as Gemini generation — no additional credentials needed.

Document version 2.0 — RAG architecture. Freeze API contract (Section 7) before Day 1 standup.